

API SECURITY RISK ANALYSIS REPORT

Internship Task 3 — Cyber Security Domain
Future Interns Program

Submitted By:	Satyam Kumar
CIN ID:	FIT/MAY26/CS8811
Track Code:	CS
Repository:	FUTURE_CS_03
Task:	Task 3 — API Security Risk Analysis
Date:	May 2026

1. Executive Summary

APIs (Application Programming Interfaces) are the backbone of modern web and mobile applications. They enable communication between services, but insecure APIs are one of the leading causes of data breaches today. This report presents a comprehensive security risk analysis of three publicly available demo/test APIs, identifying vulnerabilities, assessing risk levels, and providing remediation recommendations.

This task was completed as part of the Future Interns Cyber Security Internship Program (Track: CS, Task 3). Tools used include Postman, Browser DevTools, and public API testing environments.

2. What is API Security?

API security refers to the practices and tools used to protect APIs from attacks and misuse. Since APIs expose application logic and sensitive data, they are frequent targets for cybercriminals. The OWASP API Security Top 10 is the industry-standard framework for identifying the most critical API risks.

OWASP API Security Top 10 (2023):

#	Risk Name	Description
1	Broken Object Level Auth	APIs expose object IDs that attackers can manipulate to access other users' data
2	Broken Authentication	Weak authentication mechanisms allow attackers to impersonate legitimate users
3	Broken Object Property Auth	APIs expose more data than needed, leaking sensitive fields
4	Unrestricted Resource Consumption	No rate limiting allows DoS attacks and resource abuse
5	Broken Function Level Auth	Users can access admin-level functions they shouldn't have access to
6	Unrestricted Access to Business Functions	Automated misuse of business logic (e.g., bulk buying, scraping)
7	Server Side Request Forgery	API fetches remote resources without validating user-supplied URLs
8	Security Misconfiguration	Default configs, open cloud storage, verbose error messages
9	Improper Inventory Management	Outdated or undocumented API versions left exposed
10	Unsafe Consumption of APIs	Trusting third-party APIs without validation

3. APIs Analyzed

Three publicly available demo/test APIs were selected for this analysis. These are intentionally vulnerable or open APIs used for educational and testing purposes.

#	API Name	Base URL	Type
1	JSONPlaceholder API	https://jsonplaceholder.typicode.com	Public REST API (Demo)
2	OWASP crAPI	https://crapi.apisec.ai	Intentionally Vulnerable API
3	Reqres API	https://reqres.in/api	Public REST API (Test)

4. Detailed API Risk Analysis

API 1: JSONPlaceholder API

Base URL	https://jsonplaceholder.typicode.com
Endpoints Tested	/users, /posts, /comments, /todos
Authentication	None
Rate Limiting	Not Implemented

Vulnerability	OWASP Category	Severity	Description
No Authentication	API2: Broken Authentication	HIGH	Any user can access /users endpoint and retrieve full user data including email addresses
Excessive Data Exposure	API3: Broken Object Property Authorization	HIGH	GET /users returns full PII (name, email, phone, address, geo-coordinates, etc.)
No Rate Limiting	API4: Unrestricted Resource Consumption	MEDIUM	Unlimited requests can be made — vulnerable to scraping, DoS, and data harvesting
IDOR Vulnerability	API1: Broken Object Level Authorization	HIGH	Accessing /users/1, /users/2 etc. exposes all user records — no ownership checks
No HTTPS Enforcement	API8: Security Misconfiguration	MEDIUM	HTTP requests are not redirected to HTTPS — data can be intercepted

Overall Risk: HIGH

API 2: OWASP crAPI (Completely Ridiculous API)

Base URL	https://crapi.apisec.ai
Endpoints Tested	/identity/api/auth/login, /community/api/v2/community/posts, /workshop/api/shop/products
Authentication	JWT Token (Weak Implementation)
Rate Limiting	Partially Implemented

Vulnerability	OWASP Category	Severity	Description
JWT Token Not Expiring	API2: Broken Authentication	HIGH	JWT tokens do not expire — a stolen token can be used indefinitely to access user data
BOLA/IDOR on Vehicle API	API1: Broken Object Level Authorization	CRITICAL	Changing vehicle ID in request allows accessing other users' vehicle data
Mass Assignment	API3: Broken Object Property Authorization	HIGH	POST /community/api/v2/community/posts accepts extra fields like 'admin' and 'password'
Coupon Abuse via No Rate Limiting	API6: Business Flow Abuse	MEDIUM	The coupon redemption endpoint has no rate limiting — same coupon can be used multiple times
Verbose Error Messages	API8: Security Misconfiguration	MEDIUM	Error responses reveal internal stack traces, database field names, and sensitive data

Overall Risk: CRITICAL

API 3: Reqres API

Base URL	https://reqres.in/api
Endpoints Tested	/users, /login, /register, /users/{id}
Authentication	Token-based (Simulated)
Rate Limiting	Not Implemented

Vulnerability	OWASP Category	Severity	Description
Fake Token Authentication	API2: Broken Authentication	HIGH	POST /login returns a token for ANY credentials without actual validation –
No Input Validation	API8: Security Misconfiguration	MEDIUM	Registration endpoint accepts any payload including empty strings, special
Predictable User IDs	API1: Broken Object Level Auth	MEDIUM	User IDs are sequential (1,2,3...) — easy to enumerate all users by iteratin
No Rate Limiting on Login	API4: Unrestricted Resource Consumption	HIGH	No brute-force protection on /login — unlimited password attempts allowed
Missing Security Headers	API8: Security Misconfiguration	LOW	Responses missing X-Content-Type-Options, X-Frame-Options, and Conte
Overall Risk: HIGH			

5. Risk Summary

API Name	Auth	Rate Limit	Data Exposure	Input Validation	Overall Risk
JSONPlaceholder	None	None	HIGH	Partial	HIGH
OWASP crAPI	JWT (Weak)	Partial	CRITICAL	Poor	CRITICAL
Reqres API	Simulated	None	MEDIUM	Poor	HIGH

6. Remediation Recommendations

- ✓ **Implement Strong Authentication:** Use OAuth 2.0 or JWT with proper expiry times (15-60 min). Never use simulated or static tokens in production. Enforce MFA for sensitive endpoints.
- ✓ **Apply Rate Limiting & Throttling:** Limit API requests per IP/user (e.g., 100 requests/minute). Use tools like AWS API Gateway, Nginx rate limiting, or API management platforms.
- ✓ **Fix IDOR / BOLA Vulnerabilities:** Always validate that the requesting user owns the resource. Never use sequential IDs — use UUIDs. Implement object-level authorization checks on every endpoint.
- ✓ **Minimize Data Exposure:** Return only the fields required by the client. Never expose PII like full address, phone, or geo-location unless absolutely necessary. Use response filtering.
- ✓ **Add Input Validation & Sanitization:** Validate all input on the server side. Reject unexpected fields (prevent mass assignment). Use allowlist validation — never rely on frontend-only validation.
- ✓ **Enable Security Headers:** Add X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security, and Content-Security-Policy headers to all API responses.
- ✓ **Suppress Verbose Errors:** Return generic error messages to clients. Log detailed errors server-side only. Never expose stack traces, DB queries, or framework details in responses.
- ✓ **API Versioning & Inventory Management:** Maintain an up-to-date API inventory. Deprecate and remove old API versions. Document all endpoints and their access controls.

7. Tools & Techniques Used

Tool	Purpose
Postman	Send API requests, inspect headers, responses, and status codes
Browser DevTools (Network Tab)	Capture and analyze API calls made by web applications
jwt.io	Decode and inspect JWT tokens for weak signing and missing claims
MXToolbox / SecurityHeaders.com	Check security headers and DNS configurations
OWASP API Security Top 10	Framework for identifying and classifying API vulnerabilities
crAPI (apisec.ai)	Intentionally vulnerable API used for hands-on testing practice
GitHub	Public repository for documentation and submission

8. Conclusion

This report analyzed three APIs — JSONPlaceholder, OWASP crAPI, and Reqres — against the OWASP API Security Top 10 framework. All three APIs exhibited significant security vulnerabilities including broken authentication, missing rate limiting, excessive data exposure, and IDOR vulnerabilities.

API security must be treated as a first-class concern during development. Developers should conduct regular security audits, follow the principle of least privilege, implement proper authentication and authorization, and validate all inputs. The findings in this report demonstrate that even simple APIs can pose serious security risks if not designed and tested with security in mind.
